

Technische Betriebsanleitung

Tensor — Interactive ONNX Studio

Lokaler Betrieb · React · ONNX · Browser-Speicher

Stand: 22.08.2026

Inhalt

1	Zweck und Einordnung	1
2	Systemübersicht	1
3	Voraussetzungen	1
4	Betriebsablauf	1
4.1	Material, Form und Impuls	1
4.2	Codex	1
4.3	Flow Lens, Lernkarte und Sensorik	1
5	ONNX-Import und Export	2
6	Lokaler Beispielspeicher und Compose	2
7	Sicherheits- und Datenschutzbetrieb	2
8	Störungssuche	2
9	Referenzen	2

1 Zweck und Einordnung

Tensor ist ein lokales Browser-Studio für das verständliche Gestalten kleiner ONNX-Tensorstrukturen. Die Anwendung übersetzt Material, Form, Werte, Operation, Codex und Graphroute in einen nachvollziehbaren Ablauf. Tensor ist keine Trainingsplattform und keine serverseitige Inferenzumgebung; es ist eine zugängliche Brücke zwischen mathematischer Struktur und exportierbarem ONNX-Artefakt.

Tensor verarbeitet Werte, ONNX-Dateien und Codex-Fragmente im Browser. Wichtige Ergebnisse werden als ONNX-Datei oder SVG-Lernkarte auf dem Gerät gesichert.

2 Systemübersicht

| Funktion | Ausführung | Persistenz | |—|—|—| | Shape, Werte und Operation | Browser | Aktuelle Sitzung
| | ONNX-Import | Browser | Keine Serverkopie | | ONNX- und Lernkarten-Export | Browserdownload |
Gerät des Nutzers | | Codex | Browser | Aktuelles oder gespeichertes Modell | | Mini-Modelle | Browser-
localStorage | Browserprofil | | Sensorik | Browser nach Freigabe | Nicht persistent |
WebAssembly kann zukünftig als lokaler Rechenkern dienen. MDN beschreibt Wasm als binäres Ziel für performante Browseranteile, das bewusst neben JavaScript arbeitet. [1]

3 Voraussetzungen

Ein aktueller Chromium-, Firefox- oder Safari-basierter Browser wird empfohlen. Für die kompakte Mobile-Ansicht genügen 375 Pixel Breite. Die zweispaltige Studioansicht profitiert von Desktop oder Tablet. WebGPU ist optional; die Canvas-Vorschau dient als Fallback. Gerätesensorik ist optional und wird erst nach bewusster Nutzerfreigabe aktiviert.

4 Betriebsablauf

4.1 Material, Form und Impuls

Der Standardfluss lautet **Material** → **Form** → **Impuls** → **Codex** → **Graph**. Material bestimmt den Datentyp float32, int32 oder bool. Form bestimmt Rank, Shape und Wertevolumen. Impuls bestimmt die ONNX-Operation. Direkte Werte müssen exakt zur Shape passen. Für geeignete Matrizen steht ein Heatmap-Grid bereit.

4.2 Codex

Der Codex nimmt lokale Text-, JSON- und SVG-Fragmente bis zur vorgesehenen Größenbegrenzung auf. Beim Export werden sie als ONNX-Metadaten integriert. Der Codex ist kein Dokumentenspeicher; große oder vertrauliche Inhalte gehören nicht in ein ONNX-Modell.

4.3 Flow Lens, Lernkarte und Sensorik

Die Flow Lens zeigt Input, Operation und Output als lesbare Route. Die Leinen sind visuelle Kanten eines Graphen, keine physikalische Aussage über Gewichte oder Trainingsdynamik. Die Route kann als lokale SVG-Lernkarte gesichert werden. Maus und Touch bewegen die Ansicht unmittelbar; Sensorik ist ein freiwilliger Zusatz.

5 ONNX-Import und Export

Tensor liest lokale .onnx-Dateien bis zur vorgesehenen Importgrenze und zeigt Graph, Operationsnamen, Metadaten, Shape und – bei unterstützten Datentypen – Werte. Nicht jede ONNX-Variante ist vollständig editierbar. Für eine breitere, spezialisierte Modellinspektion eignet sich Netron als ergänzender Viewer. [2] Der ONNX-Export erzeugt eine Datei im Browser. Vor einem Release sind ein Round-Trip-Test, eine TypeScript-Prüfung und ein Produktions-Build auszuführen.

| Prüfung | Befehl | Erwartung | |---| | Typprüfung | `pnpm check` | Keine TypeScript-Fehler | | Round-Trip | `pnpm vitest run client/src/pages/Home.test.ts` | Test erfolgreich | | Produktions-Build | `pnpm build` | Erfolgreicher Build |

6 Lokaler Beispielspeicher und Compose

Mini-Modelle liegen unter dem lokalen Schlüssel `tensor-mini-models-v1`. Kategorien sind **Entwürfe**, **Lernen**, **Experimente** und **Sammlung**. Ein Nutzer kann speichern, laden, umkategorisieren oder löschen. Browserdaten zu löschen entfernt auch diesen Speicher; wichtige Ergebnisse sollten exportiert werden.

Compose verbindet den aktuellen Tensor mit einem gewählten zweiten Szenenmodell. Die Operation ist absichtlich konservativ: Datentyp und Shape müssen identisch sein. Direkte Zahlenwerte werden positionsweise gemittelt; Boolean-Werte werden logisch verodert. Codex-Fragmente werden zusammengeführt und doppelte Einträge unterdrückt. Das Resultat bleibt editierbar.

7 Sicherheits- und Datenschutzbetrieb

Tensor arbeitet lokal. Vor einer Veröffentlichung sind HTTPS, eine restriktive Content-Security-Policy, Fehlerbehandlung für Importdateien, ein Datenschutzhinweis zum `localStorage` und ein dokumentierter Aktualisierungsprozess für Abhängigkeiten vorzusehen.

WebAssembly ist kein Geheimtresor. Ein Browser muss ein Modul laden und ausführen können; ausgelieferter Client-Code kann daher untersucht werden. WASM eignet sich für deterministische Rechenkerne wie Compose, Shape-Prüfung oder Kennzahlen, nicht für Schlüssel, Autorisierung, Zahlungslogik oder vertrauliche Geschäftsregeln. Die detaillierte Einordnung steht in `dokuDisA.txt`.

8 Störungssuche

| Beobachtung | Erste Maßnahme | |---| | Export erscheint nicht | Shape, Datentyp und Werteanzahl prüfen | | ONNX-Import unvollständig | Modellgröße und unterstützten Datentyp prüfen; bei Bedarf in Netron gegenprüfen | | Mini-Modelle fehlen | Browserdaten, Profil oder lokalen Speicher prüfen | | Compose blockiert | Gleiche Shape und gleichen Datentyp für beide Stränge wählen | | Sensorik bewegt nichts | Freigabe prüfen; Maus- und Touch-Fallback verwenden |

9 Referenzen

- [1] [MDN – WebAssembly](#)
- [2] [Netron](#)
- [3] [ONNX Runtime Web](#)

[4] [wasm-bindgen Guide](#)

[5] [Zig Language Reference](#)